황혁주 | 경력기술서_2026

> 타겟팅 조건이 얼마나 복잡해도 serving은 0.11ms입니다. 300만 유저 × AND/OR/NOT 중첩 조건을 배치가 모두 흡수하고, 광고 요청은 단일 PK 조회 하나로 끝납니다. 이 구조를 광고 플랫폼 초기부터 직접 설계했습니다.
> DAU 100만 e-commerce에서 피크 10M req/시간을 다뤘고, 광고 장애가 홈 화면 전체로 번지던 구조를 끊었습니다. 코드만 아니라 데이터 파이프라인·모니터링·배포 자동화까지 처음부터 직접 만드는 개발자입니다.

경력

기간	회사 / 팀	역할
2025.04 ~ 현재	한국신용데이터(KCD) | 광고플랫폼팀	신규 광고 플랫폼 초기 설계 및 개발 admin · audience · serving · tracking-api 도메인
2022.06 ~ 2025.04	카카오스타일(지그재그) | 전시팀	홈·카테고리 Server Driven UI 백엔드 개발 DAU 100만, 피크 트래픽 10M req/시간

프로젝트 상세

1. Audience 타겟팅 O(log N) 서빙 아키텍처 설계

KCD 광고플랫폼팀 · 2025

배경 / 문제

신규 광고 플랫폼을 처음부터 설계하는 상황이었습니다. 광고주가 타겟팅 조건을 설정할 때 **3가지 데이터 소스**를 AND/OR/NOT으로 자유롭게 조합할 수 있어야 했습니다.

- Audience: 약 300만 명의 유저 식별자 + 성별·연령·업종 등 정적 속성.
- Dimension: 앱버전·첫구매여부·멤버십 여부 등 동적 행동 속성.
- Target Group: 광고주가 직접 업로드한 타겟팅 대상 파일.

> "복잡한 타겟팅 조건 평가(AND/OR/NOT)를 serving 시점에 수행할 것인가, batch 시점에 사전 연산해둘 것인가?"

실시간 연산 방식으로 가면 타겟팅 조건이 복잡해질수록 serving latency가 선형으로 증가합니다. 300만 명 규모에서 매 요청마다 3개 테이블의 조건 평가를 반복하면 타겟팅 로직의 복잡도와 serving 성능이 강하게 결합되는 구조적 문제가 생깁니다. 이 결합을 끊는 것이 이번 설계의 목표였습니다.

파이프라인 전체 구조

```
(1) Source Data
+-------------------------------------------+
|  CDC Catalog (ops DB replica)  PLOVER     |
|  user behavior / app version / orders     |
|                        biz info / region  |
+------------+-------------------+----------+
             |                   |
             v                   v
(2) [SP] custom_dimension_builder -- Databricks Workflow 1
+-----------------------------------------------------+
|  run_notebooks_in_directories.py (super notebook)   |
|    dimension/*.py x N  <- auto-scan on file add     |
|  build_targeting_user_meta.py    -> S3 parquet      |
|  build_targeting_business_meta.py-> S3 parquet      |
+-----------------------+-----------------------------+
                        |  Workflow 1 -> Workflow 2 auto-trigger
                        v
(3) [SP] ads-platform dimension -- Databricks Workflow 2
+-----------------------------------------------------+
|  audience_df.ipynb                                  |
|  (type, display_name, Audience vs Dimension routing) |
+-----------------------+-----------------------------+
                        |
                        v
(4) S3 delivery (4 JSON types)
    s3://ads-platform-private-bucket/
    temp/name=ads_platform_*_json/
                        |
```

```
          +--------------+  Jenkins daily cron trigger
          v              v
(5) Spring Batch
+----------------------+
|  AudienceSyncJob     |
|  (Upsert)            |
+----------+-----------+
           |
+-------------------------------------------+
|  DimensionSyncJob                         |
|  (1) Keys  -> Admin Field Catalog sync    |
|  (2) Values -> dimension value list sync  |
|  (3) Body  -> dimensions table Upsert     |
+----------+--------------------------------+
  JsonChunkProcessor (chunk=5,000)
           |
           v
(6) Operations DB
  audiences | dimensions | segments+conditions | target_groups
           |
           v
(7) SegmentMartJob (Spring Batch)
    segmentIds.forEach { id ->
        createMart(id)  // new N:N mapping INSERT
        deleteMart(id)  // old mapping delete
    }
           |
           v
    segment_mart (segment_id x audience_id_str) ~200M rows
    PRIMARY KEY (segment_id, audience_id_str) -> B-tree O(log N)
           |
           v
(8) Ad Serving
    SELECT 1 FROM segment_mart WHERE segment_id=? AND audience_id=?
    -> avg 0.11ms
```

해결 과정 — 단계별 상세

① 원천 데이터 수집 — CDC Catalog + PLOVER

CDC Catalog는 운영 DB의 복제본으로, 유저 행동 로그·앱버전·첫구매 여부·구매 경과일 등 동적 속성을 제공합니다. PLOVER PostgreSQL은 사업자 정보 DB로, 업종·지역·사업자 등록번호 등 B2B 속성을 제공합니다. 유저와 사업자의 관계는 N:N입니다. 이 N:N 관계를 식별하는 최소 단위가 Target ID입니다.

```
Target ID = user_uuid + "_" + business_public_id
```

Target ID가 Audience와 Dimension 파이프라인 전체를 연결하는 기본 집계 단위입니다.

② Databricks Workflow 1 — custom_dimension_builder

`run_notebooks_in_directories.py`(super notebook)가 `dimension/` 폴더를 스캔하여 `.py` 파일을 자동으로 찾아 실행합니다. 새 타겟팅 차원 추가 시 이 폴더에 `.py` 파일 하나만 넣으면 다음 Workflow 1 실행 시 자동 인식 (Self-Serve 설계의 출발점).

```
dimension_df = meta_df.select(
    concat(col("uuid"), lit("_"), col("business_public_id")).alias("public_target_id"),
    col("app_version").cast("string"),
    "has_first_purchase",
    "dpv124_users_created_at_days",
    # 추가 dpv* 컬럼들
).dropDuplicates(["public_target_id"])
```

③ Databricks Workflow 2 — ads-platform dimension

Workflow 1 완료 시 Databricks Workflow 의존성(A→B)에 의해 자동 트리거됩니다. `audience_df.ipynb` 의 `select()` 섹션에서 각 필드에 대해 타입(string/int/boolean)·display_name·라우팅(Audience vs Dimension)을 지정합니다. 신규 필드 추가 시 이 수정이 반드시 필요합니다 (2-step 한계 — Project 3에서 상세 설명).

④ S3 중간 적재 (JSON 4종)

Workflow 2가 생성한 JSON 4종이 S3에 적재됩니다. S3는 Databricks(쓰기)와 Spring Batch(읽기) 사이의 비동기 버퍼 역할을 합니다. Workflow 2 실패 시 이전 JSON이 그대로 남아 Spring Batch는 이전 배치의 JSON 기반으로 정상 동작합니다 (stale-but-serving의 한 축).

⑤ Spring Batch — AudienceSyncJob

Jenkins 일일 크론으로 트리거. S3의 audience JSON을 읽어 `audiences` 테이블에 Upsert합니다. `JsonChunkProcessor` : 5,000건 단위 청크 처리로 메모리 압력 제어. `BatchRetryConfig` : 최대 3회 재시도, 1s → 10s exponential backoff. 초기 DDL 기반 View Swap 패턴을 시도했으나 권한 분리 위반·race condition 문제로 폐기하고 `INSERT ... ON DUPLICATE KEY UPDATE` 기반 Upsert로 전환 → 적재 시간 **4시간 → 30분 (87% 단축)**.

⑥ Spring Batch — DimensionSyncJob (3단계, 순서 중요)

- ① Keys 단계: S3 JSON에서 dimension 필드 키 목록을 읽어 `dimensions` 테이블의 키 스키마를 동기화합니다. **이 단계에서 admin 화면용 필드 카탈로그도 함께 자동 갱신됩니다.** `.py` 파일을 추가하고 배치가 한 번 돌면 다음날 어드민 UI에 새 타겟팅 항목이 자동으로 나타나는 이유가 이 단계에 있습니다.
- ② Values 단계: 각 dimension 필드에서 가능한 값 목록(예: app_version의 경우 "3.0.1", "3.1.0" 등)을 동기화합니다.
- ③ Body 단계: Target ID별 실제 dimension 값을 `dimensions` 테이블에 Upsert합니다.

⑦ 운영 DB — 타겟팅에 사용되는 테이블 구성

- `audiences` : 약 300만 명의 유저 식별자 + 성별·연령·업종 등 정적 Audience 속성
- `dimensions` : Target ID별 앱버전·첫구매·멤버십 등 동적 Dimension 속성
- `segments + conditions` : 광고주가 admin UI에서 설정한 타겟팅 조건 (AND/OR/NOT 조합)
- `target_groups` : 광고주가 직접 업로드한 타겟팅 대상 파일

⑧ SegmentMartJob — Materialized View 패턴의 핵심

모든 segment에 대해 해당 조건(AND/OR/NOT 조합)을 만족하는 audience_id를 사전에 평가하여 `segment_mart` 테이블에 N:N 매핑으로 적재합니다.

```
segmentBatchService.retrieveAllIds().forEach { id ->
    segmentMartBatchService.createMart(id)   // 신규 N:N 매핑 INSERT
    segmentMartBatchService.deleteMart(id)   // 구 매핑 삭제 (created_at 기준)
}
```

`createMart → deleteMart` 순서가 핵심입니다. 신규 데이터를 먼저 적재한 뒤 구 데이터를 후행 삭제하여 serving 단절 없이 데이터가 교체됩니다.

```
CREATE TABLE segment_mart (
    segment_id      BINARY(16)    NOT NULL COMMENT '세그먼트 ID',
    audience_id_str VARCHAR(63)   NOT NULL COMMENT '오디언스 ID',
    created_at      DATETIME(6)   NOT NULL,
    modified_at     DATETIME(6)   NOT NULL,
    PRIMARY KEY (segment_id, audience_id_str)
) COMMENT '세그먼트 마트 정보';
```

`PRIMARY KEY (segment_id, audience_id_str)` — MySQL InnoDB는 PK를 B-tree로 관리하므로 (segment_id, audience_id_str) 쌍에 대한 조회가 트리 탐색 한 번으로 완결됩니다. 현재 `segment_mart` 규모: 약 **2억 건**.

⑨ Ad Serving — O(log N) 단일 조회

```
SELECT 1 FROM segment_mart WHERE segment_id = ? AND audience_id = ?
```

타겟팅 조건이 AND 5개든 OR 10개든 NOT이 중첩되든 **serving 코드는 이 쿼리 하나**입니다. 조건 복잡도는 배치(SegmentMartJob)가 모두 흡수하고, serving은 항상 단순 복합 PK 조회로 응답합니다. 평균 응답: **0.11ms** (구 플랫폼 ES 기반 0.22ms 대비 50% 단축)

장애 대응 설계 — Stale-but-serving

어느 단계가 실패해도 이전 상태로 fallback하여 서빙을 유지합니다.

- **Workflow 1 실패**: 이전 S3 parquet 유지 → Workflow 2는 이전 parquet 기반으로 실행
- **Workflow 2 실패**: S3 JSON 미갱신 → AudienceSyncJob / DimensionSyncJob은 이전 S3 JSON 기반으로 실행
- **AudienceSyncJob / DimensionSyncJob 실패**: DB 미갱신 → SegmentMartJob은 이전 DB 데이터 기준으로 실행
- **SegmentMartJob 실패**: segment_mart 미갱신 → serving은 이전 mart로 정상 동작

어느 배치 단계가 실패해도 광고 송출이 0이 되는 시나리오는 없도록 설계했습니다.

> ✅ **결과**
>
> - serving 응답: 복합 PK 단일 조회 O(log N) = 평균 0.11ms (구 플랫폼 ES 기반 0.22ms 대비 50% 단축)
> - 타겟팅 조건 복잡도(AND/OR/NOT/IN/중첩)와 serving latency 구조적 분리 달성
> - 원천 데이터 적재: 4h → 30분 (87% 단축 — DDL View Swap 폐기 후 Upsert 전환)
> - 유저 속성: 200개 → 38개 (81% 경량화 — 쿼리 로그 전수 분석 후 실사용 속성만 추출)
> - segment_mart 2억 건 규모 무중단 운영 (createMart → deleteMart 순서로 serving 단절 없는 데이터 로테이션)
> - 신규 타겟팅 dimension 추가 시 serving 코드 수정 없이 배치 확장만으로 처리 가능한 구조

---

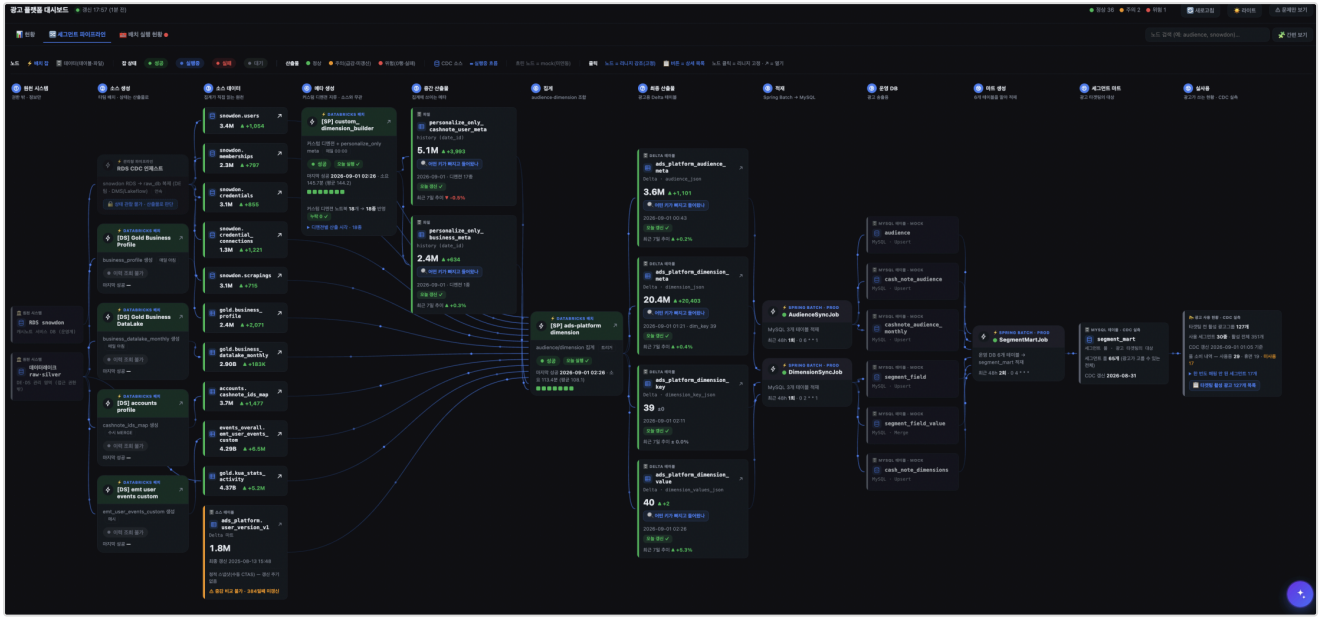
2. Audience/Dimension 파이프라인 모니터링 대시보드

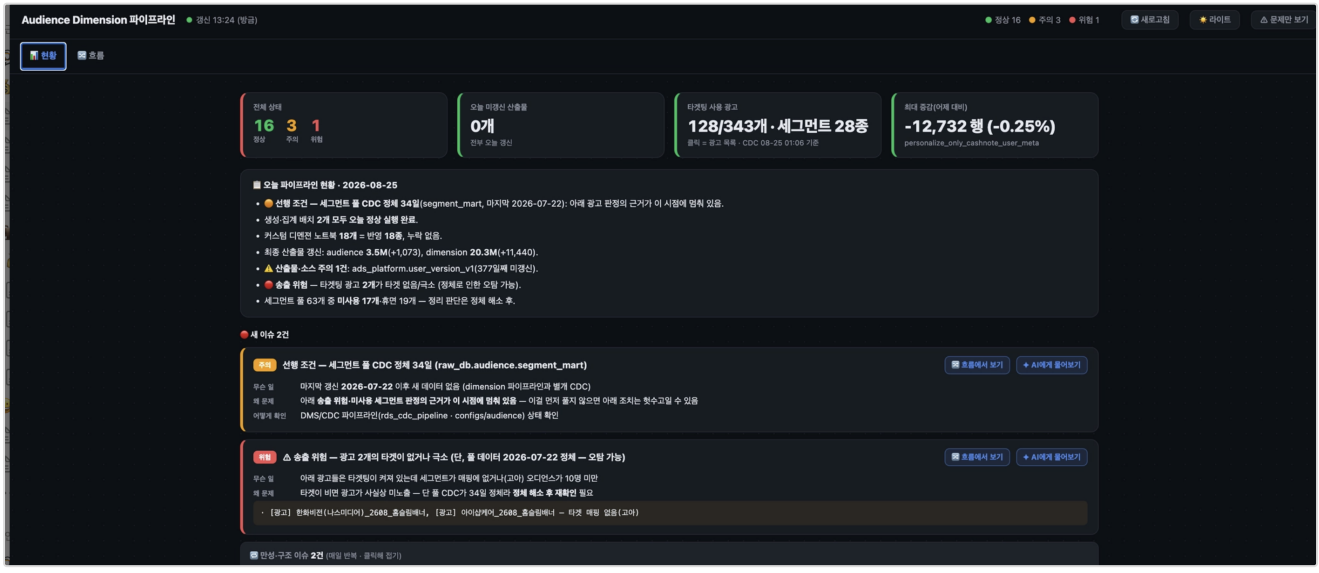
> KCD 광고플랫폼팀 · 2025~2026

배경 / 문제

Project 1에서 구축한 파이프라인은 잘 동작했지만, **파이프라인이 이상해졌을 때 어느 단계에서 무엇이 잘못됐는지 파악하기 어렵다**는 문제가 있었습니다. 원천 11개, 여러 팀의 Databricks 배치, Spring Batch, 운영 DB까지 10개 단계가 이어지는 구조에서 세 가지 문제가 눈에 안 보였습니다.

- **잡은 초록불, 데이터는 어제자**: 배치가 성공하는 것과 데이터가 갱신되는 것은 다른 말입니다. 입력이 비어 있어도, 조건에 걸려 아무것도 쓰지 않아도 잡은 성공으로 끝납니다.
- **지연 소비**: 집계 잡이 소스 마트보다 먼저 실행되면 어제 데이터를 읽고 성공으로 끝납니다. 잡도 성공, 행수도 그럴듯해서 의심하기 전까지는 드러나지 않습니다.
- **사람이 매번 확인해야 함 — 그래서 사실상 안 했음**: 원천 11개와 배치, 산출물을 매일 손으로 확인하는 일은 현실적으로 지속되지 않습니다. 문제를 발견하는 시점이 대개 "광고가 이상하다"는 제보 이후였습니다.





해결 과정 — 모니터링 대시보드 구축

구현 환경: Python 표준 라이브러리 + databricks-sdk. 외부 패키지 1개, 빌드 단계 없음. `python server.py` 한 줄로 실행.

10열 리니지 뷰 — 원천부터 광고 송출까지 한 화면

```
(1)Source  (2)Generated  (3)Data     (4)Meta    (5)Intermediate
 [ref]      [ref]         [measured] [measured] [measured]
                                                     |
                                                     v
(6)Aggreg  (7)Final    (8)Load    (9)OpsDB  (10)Production
[measured] [measured] [n/a]      [n/a]      [measured/CDC]
```

각 열에 "이 구간을 어떻게 조회해야 하는지" 한 줄씩 명시. 미연동 구간의 색으로 상태를 판단하면 안 된다는 사실을 화면에 명시 — "못 보는 구간을 못 봤다고 적어두는 것도 감시 도구의 일" 원칙 적용.

수집-표시 분리 아키텍처

```
Background collector (every 2min, 14 threads)
  Databricks SQL: row count, 24h delta, last update time
  Databricks Jobs API: job status, start/end time
  Ad ops data: segment active/dormant/unused status
  -> completes in 15~30s -> stored in memory as "current state"

Browser (polling every 15s)
  receives full current state and re-renders
  does NOT query Databricks directly -> zero extra load
```

브라우저를 여러 개 열어도 Databricks 조회 부하는 **2분에 한 번**으로 고정.

이상 감지 기준

- 어제 대비 **10% 이상 감소** → 주의 (정상 일간 변동이 1% 미만이라 10%는 10배 한계)
- 어제 대비 **30% 이상 급변**(양방향 무관) → 주의
- **3일 이상 미갱신** → 주의 (테이블마다 기준일 별도 설정 가능)
- 평소 완료 시각(7일 중앙값) + **90분 경과 후 미갱신** → 주의 (고정 시각 안 씀 — 파이프라인마다 평소를 스스로 학습)
- **0행 또는 잡 실패** → 즉시 위험
- **송출 위험**: 활성 광고의 세그먼트가 풀에 없거나(ghost) 오디언스 10명 미만 → 위험

설계 원칙

- **비율 기반 실측**: 배치 성공 여부로 데이터 신선도를 추정하지 않습니다. 24시간 전 시점 테이블을 Delta Time Travel로 직접 읽어 증감을 비교하고, 마지막 갱신 시각은 변경 이력에서 가져옵니다.
- **원인을 결과 위로**: 근거 데이터가 없으면 해당 판정 여백에 "오탐 가능"을 박아 넣습니다.
- **급성 vs 만성 분리**: 매일 반복되는 구조적 이슈는 접어두고, **오늘 처음 생긴 문제만** 펼쳐서 보여줍니다.
- **사람 말로 3줄 고정**: 이슈 카드 = "무슨 일 / 왜 문제 / 어떻게 확인하나".
- **못 본 것은 못 봤다고 말한다**: 조회 실패 시 무음으로 변환하지 않고 정확한 오류를 표시.

화면을 통해 실제로 발견된 것들

- **세그먼트 풀 데이터 34일 정체**: 세그먼트 미사용/휴면/송출위험 판정의 근거 데이터가 한 달이 넘도록 정체됐는지도 모르고 있었습니다. 정체가 풀리자 송출위험 경보 2건이 0건이 됐습니다(오탐 확인). "오탐 가능"을 판정 여백에 미리 적어둔 덕분에 잘못된 조치를 막았습니다.
- **세그먼트 사용 현황 가시화**: 전체 64개 중 사용중 28, 휴면 19, 한 번도 미사용 17. 이전에는 "세그먼트가 몇 개 있다"는 것만 알았고 실제로 쓰이는지는 아무도 답할 수 없었습니다.
- **자가 진단**: 코드를 6개 관점으로 진단 → 75건 (잘된 결정 16, 결함 44, 개선 여지 15). 핵심 결함: 조회 실패 시 빈 값으로 바꾼 후 "이상 없음"으로 판정 → 9곳 수정.

현재 한계: 알림 없음 — 화면을 열어야만 확인 가능 (팀 공유 단계에서 Slack 알림 연동 예정)

> ✅ **결과**
>
> - "광고가 이상하다"는 제보 이후 발견 → 10열 리니지에서 어느 단계에서 문제가 생겼는지 즉시 확인 가능한 구조로
> - 배치 성공 ≠ 데이터 신선도라는 사각지대 해소 (실측 원칙)
> - 세그먼트 풀 34일 정체 등 기존에는 볼 수 없었던 이슈 자동 감지
> - 중앙값 기반 경보로 고정 시각 기준의 낡은 형태의 알림 제거
> - 광고 데이터와 파이프라인 데이터를 겹쳐 보는 송출 위험 감지 (360개 활성 광고 중 130개가 28종 세그먼트 사용, 현재 0건 위험)

---

3. Self-Serve 타겟팅 차원 확장 파이프라인

KCD 광고플랫폼팀 · 2025

배경 / 문제

마케터가 특정 타겟팅 조건을 새로 요청할 때마다, 데이터팀 소관인 user_meta·business_meta에 해당 컬럼을 추가하는 절차가 필요했습니다. 데이터팀 협의 → Databricks 노트북 수정 → Spring Batch 파싱 검증까지, 단계가 복잡하고 시간이 걸렸습니다. 정형 메타는 데이터팀이 계속 유지하되, **타겟팅 전용 커스텀 차원만 빠르게 추가할 수 있는 별도 경로**가 필요했습니다.

Self-Serve 파이프라인 흐름

```
Step 1  Add new_dimension.py to dimension/ folder        [manual]
          |
```

```
         v
Step 2  Workflow 1: auto-scans .py files and executes    [automatic]
         |
         v
Step 3  Workflow 2: audience_df.ipynb -> S3 JSON output
        (requires select() update -- see Runbook)         [auto w/ caveat]
         |
         v
Step 4  DimensionSyncJob (1) Keys step executes          [automatic]
        -> new dimension key auto-registered to admin Field Catalog
         |
         v
Step 5  Next morning: new targeting item shown in admin UI [automatic]
```

DimensionSyncJob ① Keys 단계의 역할

Self-Serve의 핵심은 DimensionSyncJob의 ① Keys 단계에 있습니다. 이 단계는 `dimensions` 테이블의 키 스키마를 동기화하는 것에서 그치지 않고, **admin 화면용 필드 카탈로그도 함께 갱신**합니다. 광고주가 타겟팅 조건을 설정하는 admin UI는 이 카탈로그를 읽어 선택 가능한 타겟팅 항목 목록을 표시합니다.

실제 한계 — 2-step 구조

- **Step 1 (자동)**: `dimension/` 폴더에 `.py` 파일 추가 → Workflow 1이 자동 실행
- **Step 2 (수동 필요)**: `audience_df.ipynb`의 `select()`에 해당 컬럼 추가 — 타입·display_name·Audience/Dimension 라우팅을 여기서 지정

Step 2를 빠뜨리면 Workflow 2가 새 dimension 컬럼을 S3에 출력하지 않아 이후 전체 파이프라인이 새 차원을 인식하지 못합니다. 이 한계를 알고 설계한 구조이며, **데이터팀 의존성을 개발팀 내부에서 처리 가능한 수준으로 내재화**하는 것이 목표였습니다. 두 군데를 모두 수정해야 한다는 사실은 Runbook에 명시했습니다.

Runbook 작성: 파이프라인 복잡도가 높아 운영 중 장애 시 빠른 대응이 어렵다는 문제가 있었습니다. **31페이지 분량의 Runbook**을 작성하여 각 배치 수동 재실행 방법, 장애 시나리오별 체크리스트, 신규 dimension 추가 절차, 알려진 이슈 목록을 문서화했습니다.

> ✅ **결과**
>
> - 신규 dimension 정의 → 어드민 화면 노출: 데이터팀 의뢰 없이 개발팀 단독으로 처리 가능한 구조 확립
> - DimensionSyncJob ① Keys 단계의 Admin Field Catalog 자동 갱신으로 별도 배포 없이 어드민 UI 반영
> - 파이프라인 전체 Runbook(31p) 작성으로 온보딩 시간 및 장애 대응 시간 단축
> - 마케터 요청 기반 타겟팅 차원 확장을 데이터팀 협의 없이 개발팀 단독으로 처리 가능한 구조 확립 (2-step은 남아 있지만 기존 정형 메타 파이프라인과 완전히 분리)

---

4. 무인 배포 파이프라인 (Deploy Pilot)

KCD 광고플랫폼팀 · 2026

배경 / 문제

광고 플랫폼 팀의 prod 배포는 주 1회 사이클로 진행됩니다. 릴리즈 PR 생성, CI 대기, 포털 승인, 이미지 빌드, ArgoCD 배포, 릴리즈 노트 작성까지 단계 수는 많지 않습니다. 문제는 단계 수가 아니라 **단계 사이의 대기**였습니다. CI는 10분쯤 걸리고, 포털 승인은 관리자가 언제 누를지 모릅니다. GitHub, DevOps Portal, ArgoCD, Slack을 오가며 상태를 눈으로 확인하는 일이 반복됐고, 순서를 건너뛰거나 서비스를 하나 빠뜨리는 실수가 나올 여지도 있었습니다. **자동화의 목표는 단계를 없애는 게 아니라 대기 구간에서 사람을 떼어내는 것**이었습니다.

파이프라인 구조

```
PHASE 0  Pre-check: auth and worktree init.
         If blocked, deployment does not start.                  [auto]
PHASE 1  Release PR: confirm deploy order, include checklist.    [auto]
PHASE 2  CI wait: on failure, send log to LLM -> analyze ->
         fix code -> re-push, up to 5 retries.                   [auto]
PHASE 3  Approval request + Slack alert / await portal approval. [auto -> human]
PHASE 4  Image build+push (all services parallel) / ArgoCD.      [auto -> human]
PHASE 5  Canary check + post-deploy monitoring (Datadog, off).   [designed]
PHASE 6  Release publish and wrap-up.                            [button]
```

사람이 개입하는 지점은 **포털 승인 하나**. 나머지는 되돌리기 비싼 세 지점(카나리 승격·master 머지·릴리즈 발행)만 의도적으로 남겨 뒀습니다.

핵심 설계 결정

① 대시보드와 실행 세션을 별도 프로세스로 분리

CI 10분, 승인 대기 몇 시간처럼 기다림이 본체인 작업에서 한 덩어리로 묶으면 대기 내내 화면까지 붙잡혀야 하고, 하나가 죽으면 둘 다 죽습니다. 대시보드(ui/server.py)와 헤드리스 실행 세션을 분리하면 브라우저를 닫아도 배포는 계속되고, 세션이 죽어도 화면은 남습니다.

② 단일 파일 접점 (state.json)

두 프로세스가 소켓도, 메시지 큐도, 공유 메모리도 없이 파일 하나만 공유합니다. 세션이 단계마다 상태를 쓰고 화면이 5초마다 읽어 렌더링합니다. 접점을 파일 하나로 좁힌 덕에 세션이 죽어도, 노트북을 재부팅해도 진행 상태가 디스크에 남고, 다시 켜면 어디까지 갔는지 그대로 읽힙니다.

③ 워치독 — 죽는 것을 전제로 설계

헤드리스 세션이 죽는 것은 예외가 아니라 정상 동작으로 전제했습니다. 상태 파일이 15분간 갱신되지 않으면 세션이 멈춘 것으로 보고 **처음부터가 아니라 중단된 단계부터** 재시작합니다. 재시도 간격은 30초에서 시작해 실패마다 2배씩 늘려 최대 5분, 60회까지 시도합니다.

진단 중 발견된 틈: 빌드를 요청한 직후 식별자를 상태 파일에 적기 전에 세션이 정리되면 같은 빌드를 두 번 트리거할 수 있습니다. 재개 규칙이 세 곳에 조금씩 다르게 적혀 있는 것이 원인 — 문서화 후 수정 대기 중.

④ 워크트리 격리

파이프라인의 git 작업(릴리즈 브랜치 생성, CI 수정 커밋 등)을 전용 워크트리에 격리합니다. 배포가 돌고 있는 동안에도 내 작업 디렉토리는 브랜치와 파일이 그대로입니다.

⑤ 있는 것을 부른다

GitHub·ArgoCD·Slack은 사내에 이미 CLI와 셸 스크립트가 있습니다. 같은 일을 새로 감싸지 않고 있는 것을 직접 부릅니다. 자격증명은 전부 홈 디렉토리 파일에서 읽고 레포에도 로그에도 값이 남지 않습니다. 실패했을 때 사람이 같은 명령을 터미널에 그대로 쳐서 재현할 수 있는 것도 이 이유입니다.

⑥ LLM 기반 CI 자동 수정 — 에이전트 루프

빌드 실패 시 로그를 LLM에 전달하고, 분석 결과를 바탕으로 코드를 수정·커밋·재푸시하는 루프를 최대 5회 반복합니다. 사람이 로그를 확인하고 수정해서 재트리거하는 구간을 에이전트가 대신합니다. 5회 내에 통과하지 못하면 상태를 FAILED로 기록하고 Slack으로 알립니다.

자가 진단

프로덕션을 건드리는 도구를 만들었으면 그 도구도 검증해야 합니다. 스킬 본문·서버·화면·설정·워크플로를 **6관점(배포 안전성·아키텍처·코드 품질·장애 대응·조작 흐름·보안)**으로 나눠 진단하고, 지적마다 실제 코드로 반증을 시도해 살아남은 것만 남겼습니다.

- **잘된 결정 21건**: "판정 불가"를 통과로 취급하지 않는 원칙 관철, 안전장치를 화면이 아닌 서버에 배치, 프로세스 분리로 런이 브라우저 수명에 묶이지 않음 등
- **결함 49건**: CI 자동 수정이 공용 브랜치에 리뷰 없이 push됨 / Slack 발송 실패 시 조용히 삼키는 구조 / 재개 규칙 세 곳이 달라 빌드 중복 실행 가능성 등
- **즉시 조치**: 레포 내 다른 워크플로 파일에 Slack webhook 주소가 평문으로 박혀 있던 것을 발견하여 폐기 후 재발급 완료

> ✅ **결과**
>
> - 배포 리드타임: 3시간 → 1시간 (버튼 1회, 이후 Slack 알림만 받으면 됨)
> - "시간의 총량이 아니라 종류가 바뀌었다" — 사람이 점유당한 3시간에서 대체로 비어 있는 1시간으로
> - 실사용 23건 운영 (2026-08-11 ~ 08-27, 데모 9건 제외, 전체 32건)
> - 배포 순서를 GitHub Actions 게이트로 강제 — 로컬 도구가 폭주해도 서버에서 막히는 구조

---

5. SDUI 2-Phase 아키텍처 개편

카카오스타일(지그재그) · 2024

배경 / 문제

전시팀 백엔드 담당으로 2-Phase 아키텍처 분리 구조를 직접 설계하고 구현했습니다.

지그재그 홈·카테고리는 Server Driven UI 구조. 화면 전체 데이터를 단일 GraphQL 쿼리로 가져오는 방식을 수년 운영하며 3가지 문제가 누적됐습니다.

- **Head-of-Line Blocking**: 가장 느린 컴포넌트에 전체 로딩 시간이 종속. P90 670ms
- **장애 전파**: 일부 컴포넌트 에러 → 화면 전체 blank
- **확장성 한계**: 신규 컴포넌트 추가 시 거대 단일 쿼리를 수정해야 하는 의존성

해결 과정

- **Phase 1 — template 쿼리**: 컴포넌트 최소 정보(type, id)만 즉시 반환 → 클라이언트가 스켈레톤 UI 즉시 렌더링 (P90 4~5ms)
- **Phase 2 — Lazy Loading**: 각 컴포넌트 type에 대응하는 API를 독립 호출. 컴포넌트 간 데이터 의존성 완전 제거
- **Fault Isolation**: 특정 컴포넌트 실패가 화면 전체로 전파되지 않는 구조. 부분 렌더링 보장
- 사내 기술 블로그 게재 (카카오스타일 개발 블로그 2024.08)

✅ 결과

- ATF 렌더링: P90 670ms → 10ms (98% 단축)
- Fault Isolation: 컴포넌트 단위 부분 렌더링 보장
- 확장성: 신규 컴포넌트 추가 = 독립 API 추가만으로 완결

---

6. 광고·홈피드 리팩토링 (Fault Isolation + 성능 개선)

카카오스타일(지그재그) · 2023~2024

배경 / 문제

전시팀 백엔드 담당으로 리팩토링 전략 수립부터 Fault Isolation 구조 설계, 비동기 병렬 처리 전환까지 직접 주도했습니다.

광고 서비스(AD)가 UI 구성까지 담당하는 강결합 구조. AD API가 홈 화면 응답 전체를 만들어주다 보니 AD 장애 = 홈 화면 장애로 직결. 응답 시간은 1~1.1초.

- AD API 장애 → 홈 화면 전체 blank (Fault 전파, 매출 직격)
- 외부 API 순차(Sequential) 호출로 불필요한 지연 누적
- 거대 if-else 레거시 코드 (깨진 유리창), N+1 문제, 순환 호출 구조 (api2 → ux → ad → ux) 존재

해결 과정 (3단계)

- **Step 1 — 리팩토링 선행**: 거대 if-else → Factory/Strategy 패턴. ComponentFactory(컴포넌트 생성) + PagePostProcessor(페이지별 후처리) 책임 분리. PageInfoInput data class 도입으로 파라미터 14개 폭발 문제 해결
- **Step 2 — Fault Isolation**: UI 구성 책임을 AD → UX로 이관. AD는 상품 ID만 제공. 광고 장애가 홈 화면으로 전파되지 않는 구조
- **Step 3 — 비동기 병렬 처리**: Kotlin Coroutines `concurrentIO` 로 순차 → 병렬 전환. N+1 fetchJoin 해결, 순환 호출 구조 제거
- **무중단 배포**: AB테스트 5% → 단계적 확대, Datadog 모니터링 병행 (4~5일에 걸쳐 100%)

✅ 결과

- 응답 속도: 1.1s → 850ms (약 23% 개선)
- Fault Isolation: 광고 API 장애 시에도 홈 화면 정상 렌더링 보장
- UI 통제권 회복: UI 구성 로직이 UX 서비스로 완전 이관
- 유지보수성: 신규 컴포넌트 추가 시 기존 코드 수정 불필요 (Strategy 패턴)

기술 스택

Language	Kotlin, Java
Framework	Spring Boot, Spring Batch, DGS (GraphQL), JPA
DB / Cache	MySQL, Redis, Elasticsearch, Ehcache, Caffeine
Infra / DevOps	Docker, Datadog, Databricks, AWS S3, ArgoCD, GitHub Actions
Test	Kotest (BehaviorSpec), MockK
Architecture	Hexagonal Architecture, DDD